Line 4 — Crane Batch Runner in TIA Portal (UDT → DB → FBs → OPC UA handshake)

Goal: build the **PLC side** of the QR-recipe crane flow you already have in the app. The software scans a QR, generates a task list (crane, tank, dwell time, prev, next, status), and clicks **Send to PLC**. The PLC receives the batch over **OPC UA** (the same generic client we proved with OpenPLC and the S7-1500 sim — see `docs/tia_opcua/tia-plcsim-opcua-setup.md`), runs the crane through the sequence, and raises a **done** bit. The app watches that bit: while the batch runs the **Send** button is disabled and shows "*Tasks in progress*"; when **done** flips, it shows "*Tasks done*" and re-enables **Send** for the next recipe.

> **Prerequisite:** you already stood up the S7-1500 (PLCSIM Advanced) OPC UA server per `docs/tia_opcua/tia-plcsim-opcua-setup.md`. This doc reuses that same project/CPU/endpoint — we only add new blocks. Read that runbook first if the sim isn't running yet.

What you build here (4 things):

1. `typeTask` — a User-Defined Type (UDT): one task = one row of your app's table.
2. `Line4Data` — a global DB holding the task **array** + the **handshake** bits + the crane's **I/O mirror**. This is what OPC UA reads/writes.
3. `TravelTo` (FC) + `CraneSequencer` (FB) — the logic. The FB is a state machine that walks the task array; the FC is the small "which way do I move" helper it calls.
4. **OB1 (Main)** calls `CraneSequencer` once per scan.

Part 0 — The data contract (read this first, it ties both sides together)

The app's task table has these columns: **CRANE, TANK, TIME (S), PREV, NEXT, STATUS** (see the screenshots). A PLC works in **numbers**, not strings — so the crane is driven by a **station index**, not the tank *name*. Fix the physical layout to a numbered axis:

Station index	Place	Notes
`0`	**Parking**	crane's home; where it sits idle
`1`	**Loading Area**	lower here, load material, hold button
`2`	**Tank 1** (`1-1A`)	first process tank
`3`	**Tank 2** (`1-1B`)	
`4`	**Tank 3** (`1-2`)	
…	more tanks	add as the real line grows

So `station = tank position + 1` (Loading takes slot 1, tanks start at 2). Every station has a **limit switch** on top (`ls_at[station]`) that tells the crane "you have arrived here".

⚠ Station location ≠ task order — the two are independent

Keep these apart or the whole thing gets confusing:

- `station` is a tank's **fixed physical location** on the rail (where its limit switch is). Tank `1-1A` is *always* station 2, `1-1B` always 3, `1-2` always 4 — this never changes.
- **Task order** is the order rows appear in the task list (step 1, 2, 3 …). This is the **execution order** and it can jump around physically. A recipe is **not** guaranteed to run low-to-high tank: it might start at tank 3, then go back to tank 1.

The crane runs the tasks **in the order they sit in the array** — `tasks[1]`, then `tasks[2]`, then `tasks[3]` — and for each it travels to *that task's* `station`. Because `TravelTo` compares `target` vs the crane's current station and picks **forward or reverse**, the crane **automatically moves backward** when the next task is behind it. Nothing is assumed to be adjacent or ascending.

Worked example — a non-sequential recipe `1-2=5;1-1A=6` (do tank 3 first, then tank 1):

Array slot	station	means	crane action
`tasks[1]`	`4`	tank 3 (`1-2`), dwell 5 s	from Loading (1) → **forward** to 4, dip 5 s
`tasks[2]`	`2`	tank 1 (`1-1A`), dwell 6 s	`2 < 4` → **reverse** back to 2, dip 6 s

It physically passes station 3 on the way back but **ignores** it — the state machine only stops when `ls_at[target]` (station 2) is made. So reversing and skipping tanks is already handled.

The one rule that makes this work: the app must write the tasks into `tasks[1..N]` in the **same order they appear in the table** (step order). Array order *is* execution order.

The app sends, per task `i` (1..count): the target `station`, the `dwell_s` (your TIME column), and optionally `tank_name`/`crane` for display. Then it sets `task_count` and pulses `cmd_send`. The PLC runs, then answers with `sts_busy` / `sts_step` / `sts_done`.

App column	PLC field (`typeTask`)	Type	Direction	Meaning
TANK → station	`station`	`Int`	app → PLC	where the crane travels (`tank number + 1`)
TIME (S)	`dwell_s`	`Int`	app → PLC	seconds to dip/dwell in that tank
CRANE	`crane`	`String[4]`	app → PLC	display only, e.g. `7A`
TANK	`tank_name`	`String[10]`	app → PLC	display only, e.g. `1-1A`
PREV	`prev`	`Int`	app → PLC	traceability only (a pointer, as a tank number)
NEXT	`next`	`Int`	app → PLC	traceability only (a pointer, as a tank number)
STATUS	`status`	`Int`	**PLC → app**	`0` pending · `1` running · `2` done (per task)

> **Why `prev`/`next` are "traceability only" — NOT the same as physical order.** Your app emits the tasks **already in execution order** (step 1..N) and writes them into `tasks[1..N]` in that order; the PLC simply loops `idx = 1..count`, so **array position already encodes "what's next"**. The `next`/`prev` columns

> are the same ordering expressed as pointers (the neighbour's *tank number*), carried so the DB mirrors your table and an HMI can show the chain. The crane does **not** need to read them to know where to go next — and this in no way forces low-to-high travel: `tasks[2]` can be a lower station than `tasks[1]`, and the crane reverses to reach it (see "Station location ≠ task order" above).

Part 1 — Create the UDT `typeTask`

A UDT is your "user defined data type" — declare the shape of **one** task once, then reuse it.

1. Project tree → **PLC data types** → **Add new data type**. Rename it `typeTask`.
2. Add these members (exact names — the FB and OPC UA NodeIds depend on them):

Name	Data type	Start value	Purpose
`station`	`Int`	`0`	target station index (drives motion)
`dwell_s`	`Int`	`0`	dip/dwell seconds in the tank
`crane`	`String[4]`	`''`	display, e.g. `7A`
`tank_name`	`String[10]`	`''`	display, e.g. `1-1A`
`prev`	`Int`	`0`	previous tank (traceability)
`next`	`Int`	`0`	next tank (traceability)
`status`	`Int`	`0`	0 pending / 1 running / 2 done

3. **Compile** the UDT (right-click → Compile). It must be error-free before a DB can use it.

> **Keep it mostly `Int`.** Ints are the cheapest, most reliable thing to push over OPC UA. `String` members work but are heavier and easy to get wrong — they're optional display fields here, not needed to run the crane. If you want the leanest possible bring-up, start with just `station`, `dwell_s`, `status` and add the strings later.

Part 2 — Create the global DB `Line4Data`

This single DB is the whole interface: the app writes into it, the FB reads it, the app reads the results back. **You asked for `task1`, `task2`, `task3` — that is exactly an array: `tasks[1]`, `tasks[2]`, `tasks[3]` are your task1/task2/task3.** An array is what lets the FB loop over them cleanly with one `idx`, instead of copy-pasting logic per task.

1. Project tree → **Program blocks** → **Add new block** → **Data block (DB)** → name `Line4Data` → **Global DB** → **OK**. Leave **Optimized block access ON** (OPC UA uses symbolic names).
2. Add these rows:

   Batch (the task list):

Name	Data type	Purpose
`task_count`	`Int`	how many tasks are valid this batch (e.g. 3)
`tasks`	`Array[1..20] of typeTask`	the tasks — `tasks[1]` = your task1, etc.

Handshake (app ↔ PLC):

Name	Data type	Dir	Purpose
`cmd_send`	`Bool`	W	app sets **TRUE** to run this batch; **FALSE** = ack
`sts_busy`	`Bool`	R	PLC is executing the batch
`sts_done`	`Bool`	R	batch finished (latched until `cmd_send` cleared)
`sts_step`	`Int`	R	current task index 1..count (UI progress)
`sts_state`	`Int`	R	state-machine state (debugging / HMI)

Crane I/O mirror (on the sim these are just bits you toggle; on real hardware you'd map them to `%I`/`%Q` — see Part 8):

Name	Data type	Purpose
`ls_at`	`Array[0..9] of Bool`	station limit switches — `ls_at[s]` TRUE = crane over station `s`
`ls_dip`	`Bool`	dip limit switch — TRUE when hoist is fully lowered (in tank/loading)
`ls_up`	`Bool`	upper limit switch — TRUE when hoist is fully raised
`btn_start`	`Bool`	operator start button (held 3 s at loading)
`mv_fwd`	`Bool`	travel toward higher station number
`mv_rev`	`Bool`	travel toward lower station number
`mv_up`	`Bool`	hoist up
`mv_down`	`Bool`	hoist down

3. For every row the app must **write** (`task_count`, `cmd_send`, all `tasks[].station` / `.dwell_s` / display fields), tick the DB's OPC UA columns: **Accessible from HMI/OPC UA** ✅ and **Writable from HMI/OPC UA** ✅. Without *Writable*, the app's write is rejected (`BadUserAccessDenied`). The `sts_*` and `status` fields only need **Accessible** (read).

4. **Compile** the DB.

Part 3 — Create the helper FC `TravelTo`

An **FC** (Function) is a stateless helper — perfect for "given where I am and where I want to be, which motor do I fire?" No memory needed, so it's an FC, not an FB.

1. **Program blocks** → **Add new block** → **Function (FC)** → name `TravelTo` → **Language SCL** → **OK**. (Recall from the runbook: OB1 only offers LAD/FBD, but **FCs and FBs support SCL**.)
2. In the block interface, declare:
   - **Input:** `target : Int`, `cur : Int`, `atTarget : Bool`
   - **Output:** `fwd : Bool`, `rev : Bool`
   - **Return:** `TravelTo : Bool` (the block's return value = "have I arrived?")
3. Paste into the code body:

```
// TravelTo – pick the travel direction toward a target station.
// Returns TRUE when the target limit switch is already made (arrived → stop).
#fwd := FALSE;
#rev := FALSE;

IF #atTarget THEN
    ;                       // arrived: leave both motors off
ELSIF #target > #cur THEN
    #fwd := TRUE;           // higher station number → travel forward
ELSE
    #rev := TRUE;           // lower station number → travel reverse
END_IF;

#TravelTo := #atTarget;
```

4. **Compile.**

Part 4 — Create the main FB `CraneSequencer`

An **FB** (Function Block) *has* memory (an instance DB), which is exactly what a state machine and its timers need. This block does the whole sequence you described.

1. **Program blocks** → **Add new block** → **Function block (FB)** → name `CraneSequencer` → **Language SCL** → **OK**. When you later drop it into OB1, TIA creates its instance DB `CraneSequencer_DB` automatically.

2. In the block interface (the table above the code), declare these **Static** members (they keep their value between scans — that's the FB's memory):

Section	Name	Data type	Purpose
Static	`state`	`Int`	current state (see the CASE below)
Static	`idx`	`Int`	current task index (1..count)
Static	`count`	`Int`	tasks latched at batch start
Static	`cur`	`Int`	crane's last-known station
Static	`prevSend`	`Bool`	edge-detect memory for `cmd_send`
Static	`tHold`	`TON_TIME`	3-second button-hold timer (instance)

Section	Name	Data type	Purpose
Static	`tDwell`	`TON_TIME`	per-tank dwell timer (instance)
Temp	`target`	`Int`	scratch: this scan's travel target
Temp	`arrived`	`Bool`	scratch: `TravelTo` return

> `TON_TIME` is TIA's on-delay timer type. Declaring `tHold`/`tDwell` as **Static** members gives each its own instance memory — no separate timer DBs to manage.

3. Paste this into the code body. It's long but it's just the state list from your description, one `CASE` branch per state:

```
// CraneSequencer — walk the Line4Data.tasks[] batch through the crane sequence.
// Call once per scan from OB1. All shared data lives in global DB "Line4Data".
//
// Station map: 0=Parking  1=Loading  2..N = tanks (tank number + 1)
// States: 0 IDLE · 10 GO_LOAD · 20 LOWER_LOAD · 30 WAIT_BTN · 40 LIFT_LOAD
//         50 GO_TANK · 60 LOWER_TANK · 70 DWELL · 80 LIFT_TANK
//         85 GO_PARK · 90 DONE

// Publish state for HMI / debugging every scan.
"Line4Data".sts_state := #state;

CASE #state OF
    // ---------------------------------------------------------------
    0:  // IDLE — parked, hoist up, motors off, waiting for a batch.
        "Line4Data".mv_fwd  := FALSE;  "Line4Data".mv_rev  := FALSE;
        "Line4Data".mv_up   := FALSE;  "Line4Data".mv_down := FALSE;
        "Line4Data".sts_busy := FALSE;
        "Line4Data".sts_step := 0;
        // Rising edge of cmd_send with at least one task → latch & go.
        IF "Line4Data".cmd_send AND NOT #prevSend AND "Line4Data".task_count > 0
THEN
            #count := "Line4Data".task_count;
            #idx   := 1;
            "Line4Data".sts_done := FALSE;
            "Line4Data".sts_busy := TRUE;
            #state := 10;
        END_IF;

    // ---------------------------------------------------------------
    10: // GO_LOAD — travel to the Loading Area (station 1).
        "Line4Data".sts_busy := TRUE;
        #target  := 1;
        #arrived := "TravelTo"(target := #target, cur := #cur,
                               atTarget := "Line4Data".ls_at[#target],
                               fwd => "Line4Data".mv_fwd, rev =>
"Line4Data".mv_rev);
        IF #arrived THEN
            #cur := #target;
            "Line4Data".mv_fwd := FALSE; "Line4Data".mv_rev := FALSE;
```

```
            #state := 20;
        END_IF;

    20: // LOWER_LOAD — lower the hoist until the dip limit switch is made.
        "Line4Data".mv_up := FALSE;  "Line4Data".mv_down := TRUE;
        IF "Line4Data".ls_dip THEN
            "Line4Data".mv_down := FALSE;
            #state := 30;
        END_IF;

    30: // WAIT_BTN — operator loads material, then HOLDS btn_start for 3 s.
        #tHold(IN := "Line4Data".btn_start, PT := T#3S);
        IF #tHold.Q THEN            // held the full 3 seconds
            #state := 40;
        END_IF;

    40: // LIFT_LOAD — raise the hoist clear before travelling.
        "Line4Data".mv_down := FALSE; "Line4Data".mv_up := TRUE;
        IF "Line4Data".ls_up THEN
            "Line4Data".mv_up := FALSE;
            #state := 50;
        END_IF;

    // ------------------------------------------------------------------
    50: // GO_TANK — travel to THIS task's tank station (forward or reverse;
TravelTo
        //           picks the direction, so a lower next-station makes the crane
reverse).
        "Line4Data".sts_step := #idx;
        "Line4Data".tasks[#idx].status := 1;                  // running
        #target  := "Line4Data".tasks[#idx].station;
        #arrived := "TravelTo"(target := #target, cur := #cur,
                               atTarget := "Line4Data".ls_at[#target],
                               fwd => "Line4Data".mv_fwd, rev =>
"Line4Data".mv_rev);
        IF #arrived THEN
            #cur := #target;
            "Line4Data".mv_fwd := FALSE; "Line4Data".mv_rev := FALSE;
            #state := 60;
        END_IF;

    60: // LOWER_TANK — dip into the tank until the dip limit switch is made.
        "Line4Data".mv_up := FALSE; "Line4Data".mv_down := TRUE;
        IF "Line4Data".ls_dip THEN
            "Line4Data".mv_down := FALSE;
            #state := 70;
        END_IF;

    70: // DWELL — hold in the tank for this task's dwell_s seconds.
        // Convert Int seconds → Time. (Int seconds × 1000 ms.)
        #tDwell(IN := TRUE,
                PT := DINT_TO_TIME(INT_TO_DINT("Line4Data".tasks[#idx].dwell_s) *
1000));
        IF #tDwell.Q THEN
```

```
            #tDwell(IN := FALSE);            // reset the timer for the next tank
            #state := 80;
        END_IF;

    80: // LIFT_TANK — lift out, mark the task done, advance or finish.
        "Line4Data".mv_down := FALSE; "Line4Data".mv_up := TRUE;
        IF "Line4Data".ls_up THEN
            "Line4Data".mv_up := FALSE;
            "Line4Data".tasks[#idx].status := 2;            // done
            IF #idx < #count THEN
                #idx := #idx + 1;
                #state := 50;                               // next tank
            ELSE
                #state := 85;                               // all tanks done
            END_IF;
        END_IF;

    // ------------------------------------------------------------------
    85: // GO_PARK — return home to Parking (station 0).
        #target  := 0;
        #arrived := "TravelTo"(target := #target, cur := #cur,
                               atTarget := "Line4Data".ls_at[#target],
                               fwd => "Line4Data".mv_fwd, rev =>
"Line4Data".mv_rev);
        IF #arrived THEN
            #cur := #target;
            "Line4Data".mv_fwd := FALSE; "Line4Data".mv_rev := FALSE;
            #state := 90;
        END_IF;

    90: // DONE — raise the done flag, wait for the app to acknowledge.
        "Line4Data".sts_busy := FALSE;
        "Line4Data".sts_done := TRUE;
        // App acknowledges by clearing cmd_send → we reset for the next batch.
        IF NOT "Line4Data".cmd_send THEN
            "Line4Data".sts_done := FALSE;
            "Line4Data".sts_step := 0;
            #state := 0;
        END_IF;
END_CASE;

// Edge memory for cmd_send (must run every scan, after the CASE).
#prevSend := "Line4Data".cmd_send;
```

4. **Compile.** Fix any typo'd tag names (the SCL editor underlines unknown symbols).

> **Why a state machine and not one long IF:** a PLC re-runs the whole block every few ms. You can't `wait 6 seconds` in a line of code — you must *remember where you are* across scans. `#state` is that memory; each scan advances it by at most one step. This is the standard PLC pattern and it's what makes the crane move one action at a time.

Part 5 — Call `CraneSequencer` from OB1 (the "main block")

Blocks don't run unless something calls them. OB1 (`Main`) runs every scan — call the FB there.

1. Open **Program blocks → Main [OB1]**.
2. From the project tree, **drag `CraneSequencer [FB…]` onto the rung** in Network 1. TIA prompts to create the instance DB — accept the default name `CraneSequencer_DB` → **OK**. The call box appears with EN/ENO.
    - (If OB1 is SCL in your project you'd instead type `"CraneSequencer_DB"();` — but per the runbook OB1 here is LAD/FBD, so use the drag-drop call box.)
3. `TravelTo` does **not** need its own call in OB1 — `CraneSequencer` calls it internally.
4. **Enable simulation support** (still required for PLCSIM Advanced): project node `CraneSim` → **Properties → Protection →** ✅ **"Support simulation during block compilation"**.
5. **Compile → Software (rebuild all blocks)** — expect **0 errors** — then **download** to the running PLCSIM Advanced instance (Part 6 of the TIA runbook). Set **Start module** so the CPU goes **RUN**.

Part 6 — OPC UA NodeIds (what the app reads/writes)

Same rules as the S7-1500 runbook: symbolic NodeIds, namespace **3**, quotes preserved. Array elements and struct members extend the path. **Verify every NodeId with UaExpert** (or a direct `node-opcua` read) before wiring the app — the runbook warns that S7 DB tags don't always tree-browse, but a direct read of the symbolic NodeId works.

Handshake / status (scalars):

Purpose	NodeId	Type	Dir
Start / ack the batch	`ns=3;s="Line4Data"."cmd_send"`	Bool	write
Task count	`ns=3;s="Line4Data"."task_count"`	Int	write
Busy	`ns=3;s="Line4Data"."sts_busy"`	Bool	read
Done	`ns=3;s="Line4Data"."sts_done"`	Bool	read
Current step	`ns=3;s="Line4Data"."sts_step"`	Int	read

Per-task fields — the array index and member extend the string. Format (verify live!):

```
ns=3;s="Line4Data"."tasks[1]"."station"     Int   write   ← task1 station
ns=3;s="Line4Data"."tasks[1]"."dwell_s"     Int   write   ← task1 time (s)
ns=3;s="Line4Data"."tasks[1]"."status"      Int   read    ← task1 progress
ns=3;s="Line4Data"."tasks[2]"."station"     Int   write   ← task2 …
```

> **Bracket/quote format varies by firmware.** Siemens usually exposes the element as `"tasks[1]"` inside the path. If a direct read of the above returns `BadNodeIdUnknown`, open the DB in **UaExpert**, click the exact node, and copy its NodeId string verbatim — then use that shape for all indices. This is the same "read the NodeId directly, don't trust the tree" gotcha from the runbook.

DB rows the app writes must have **Writable from HMI/OPC UA** ✅ (Part 2.3), or writes fail with `BadUserAccessDenied`.

Part 7 — The handshake protocol (how "done" gets back to the app)

A clean **two-signal** handshake — no extra ack tag needed, because *clearing* `cmd_send` **is** the acknowledgement:

```
APP                                   PLC (CraneSequencer)
 │  1. write tasks[1..N].station,          state 0 (IDLE), sts_done=FALSE
 │     .dwell_s, display fields
 │  2. write task_count = N
 │  3. write cmd_send = TRUE  ──────────►  rising edge → latch count,
sts_busy=TRUE,
 │                                         run states 10…85
 │  (while busy: Send button DISABLED,
 │   UI shows "Tasks in progress",
 │   optionally poll sts_step for progress)
 │                                         all tasks done → state 90:
 │  4. poll sts_done  ◄──────────────────  sts_done=TRUE, sts_busy=FALSE
 │  5. sts_done==TRUE → toast "Tasks done",
 │     re-enable Send
 │  6. write cmd_send = FALSE ──────────►  sees cmd_send FALSE in state 90 →
 │                                         sts_done=FALSE, back to IDLE (ready
for next)
```

Rules the app must follow:

- **Disable Send** whenever `cmd_send == TRUE` **or** `sts_busy == TRUE` (i.e. from the moment you send until you've acknowledged done). Show "*Tasks in progress*".
- **Enable Send** only when `sts_busy == FALSE` **and** `sts_done == FALSE` (idle) — i.e. after you've read done **and** cleared `cmd_send`.
- Read `sts_done` on your existing 1 s PLC poll. On the rising edge of `sts_done`: toast, then write `cmd_send = FALSE`. That single write both acks the PLC and returns the UI to idle.
- Per-task `status` (0/1/2) lets you light up the STATUS column live if you poll `tasks[i].status`.

Part 8 — Software side (what to add in the app)

The backend is still a **generic OPC UA client** — no new PLC-specific code, just more read/write calls. The existing `sendCommand` in `backend/src/services/plc/plc.service.ts` writes one node; sending a batch is a **loop of those writes** in a fixed order.

Send to PLC (backend service, called by a new `POST /api/plc/send-batch` route):

1. For each generated task `i = 1..count`, **in the exact order the tasks appear in the table** (step order — this is what makes the crane run the recipe's sequence, including reversing): `writeValue("…tasks[i].station", "Int", station)` and `writeValue("…tasks[i].dwell_s", "Int",`

`dwell_s)` (+ display fields if used). `station = tank number + 1` — compute it when building the payload (see Appendix). The station is the tank's fixed location, **not** its order in the list; two different things.
2. `writeValue("…task_count", "Int", count)`.
3. `writeValue("…cmd_send", "Boolean", true)` **last** (arms the rising edge only after the data is in place).
4. Return `{ busy: true }` so the UI immediately disables **Send**.

Watch for done (extend the existing poll):

- Add `sts_busy` / `sts_done` / `sts_step` to what the PLC state poll reads.
- Frontend: disable **Send** while `cmd_send || sts_busy`; on `sts_done` rising → toast "*Tasks done*", `writeValue("…cmd_send","Boolean",false)`, re-enable **Send**.

Keep to the charter: validate the batch at the boundary (Zod: `count ≤ 20`, `station ≥ 0`, `dwell_s` within bounds, reject unknown fields), write unit/integration tests for the station-mapping + send-order, and add an e2e for the send → in-progress → done → re-enable flow.

Part 9 — Test it in PLCSIM (no real crane needed)

On the sim there's no wiring, so you **simulate the switches** by toggling the DB bits — either from a TIA **watch table** or straight from **UaExpert/the app**. Recommended: a watch table.

1. In TIA, **Watch and force tables** → **Add new watch table**. Add: `Line4Data.cmd_send`, `task_count`, `sts_state`, `sts_busy`, `sts_done`, `sts_step`, `ls_at[0]`…`ls_at[4]`, `ls_dip`, `ls_up`, `btn_start`, and `tasks[1].station` / `tasks[1].dwell_s`.
2. Go **online** (monitor). Seed a batch by hand first (later the app does this): `tasks[1].station=2, dwell_s=6`; `tasks[2].station=3, dwell_s=3`; `tasks[3].station=4, dwell_s=5`; `task_count=3`. (That's your screenshot recipe `1-1A=6;1-1B=3;1-2=5;`.)
3. Set `ls_at[0]=TRUE`, `ls_up=TRUE` (crane starts parked & raised). `sts_state` should read `0`.
4. **Modify `cmd_send` → TRUE.** Watch `sts_state` step to `10`. Now drive the crane by making the switches the state machine is waiting for:
    - state 10 (GO_LOAD): set `ls_at[0]=FALSE`, `ls_at[1]=TRUE` → advances to 20.
    - state 20 (LOWER_LOAD): set `ls_dip=TRUE`, `ls_up=FALSE` → advances to 30.
    - state 30 (WAIT_BTN): set `btn_start=TRUE` and **keep it TRUE for 3 s** → advances to 40.
    - state 40 (LIFT_LOAD): `ls_dip=FALSE`, `ls_up=TRUE` → advances to 50.
    - state 50 (GO_TANK, target=`tasks[1].station`=2): `ls_at[1]=FALSE`, `ls_at[2]=TRUE` → 60.
    - state 60: `ls_up=FALSE`, `ls_dip=TRUE` → 70 (DWELL). Wait ~6 s → 80.
    - state 80: `ls_dip=FALSE`, `ls_up=TRUE` → task1 `status=2`, `idx=2`, back to 50 for tank 3.
    - repeat for tasks 2 and 3, then state 85 (GO_PARK → `ls_at[0]=TRUE`) → **state 90**.
5. In state 90, `sts_done=TRUE`, `sts_busy=FALSE`. **Modify `cmd_send` → FALSE** → `sts_done` clears and `sts_state` returns to `0`. That's one full batch + acknowledge. ✅
6. Now repeat driving the switches from the **app** instead of TIA: the app writes the batch + `cmd_send`, you toggle the limit switches, and the app's **Send** button disables → shows "*Tasks in progress*" → flips to "*Tasks done*" and re-enables. That proves the full loop.

> On **real hardware** you delete the DB `ls_*`/`btn_*`/`mv_*` bits' manual use and instead map them to physical `%I` (limit switches, button) and `%Q` (motor contactors) in a small "I/O mapping" network in OB1

> (e.g. `"Line4Data".ls_dip := %I0.0;` … `%Q0.0 := "Line4Data".mv_up;`). The sequence logic doesn't change — only where the bits come from. That's the whole point of the I/O mirror in the DB.

Troubleshooting

Symptom	Cause / fix
State never leaves `0` after `cmd_send=TRUE`	`task_count` is 0, or `cmd_send` was already TRUE (no **rising** edge). Set it FALSE, then TRUE.
State stuck at 10/50 (never "arrives")	The target's `ls_at[station]` isn't set. In sim, toggle it; check `station = tank number + 1`.
Stuck at 30 (WAIT_BTN)	`btn_start` must stay TRUE for the **full 3 s**; a brief pulse won't trip `tHold.Q`.
DWELL never ends	`tasks[idx].dwell_s` is 0, or `tDwell` isn't reset — the code sets `IN:=FALSE` on exit; keep that line.
App write `BadUserAccessDenied`	DB row missing **Writable from HMI/OPC UA** (Part 2.3).
`BadNodeIdUnknown` on `tasks[1]…`	Array-element NodeId shape differs on your firmware — copy the exact string from **UaExpert** (Part 6).
`sts_done` never seen by app	App isn't polling it, or it already cleared because `cmd_send` was dropped too early. Read done *before* clearing `cmd_send`.
Compile error `DINT_TO_TIME`/`INT_TO_DINT`	Type-conversion helpers — they're standard in SCL; check spelling, or use `dwell_s` directly as `Time` if you change the UDT member type.
Download aborts "cannot be simulated"	Project → Properties → Protection → tick **Support simulation…**, rebuild all (runbook Part 5.4).

Appendix — station mapping (app side)

The app resolves a QR key to a tank (see `backend/src/services/task-generator.ts` → `resolveTank`, which gives the tank's 1-based `number`). Convert that to a PLC station when building the batch:

```
// Loading Area occupies station 1; tanks start at station 2.
// tank.number is 1-based within the line → station = tank.number + 1.
const station = tank.number + 1;   // Parking = 0 is the crane's home, not a task target
```

Carry `dwell_s = task.time`, and (optional) `tank_name`/`crane` straight through for display. `previous_task`/`next_task` map to `prev`/`next` for traceability but don't affect execution.

Definition of Done (per the charter)

- [ ] `typeTask` UDT + `Line4Data` DB compile clean; writable OPC UA flags set on app-written rows.
- [ ] `TravelTo` (FC) + `CraneSequencer` (FB) compile; FB called from OB1; sim reaches state 90 and back to 0.
- [ ] NodeIds verified live (UaExpert / direct read) before wiring the app.
- [ ] Backend batch-send + done-poll added with Zod bounds, ownership/authz, tests (unit + e2e for send→busy→done→re-enable).
- [ ] `.env` unchanged except endpoint/NodeIds if the sim moved; no secrets in code.
- [ ] Update `docs/software-flow.md` build-log + `docs/progress.md` in the same pass; link this file from the doc map in `CLAUDE.md`.